

Individual Lab (Case study 2)

Name: Gina Asang Gana

ID: 40962028

Cohort; A

QUESTION 7.24

```
create table department(  
deptNo int primary key,  
deptName varchar(20) not null,  
mgrEmpNo varchar(3)  
);
```

```
create table employee(  
empNo varchar(3) primary key,  
fName varchar(30) not null,  
lName varchar(30) not null,  
address varchar(50),  
DOB date,  
sex char(1) not null,  
position varchar(20) not null,  
deptNo int,  
foreign key(deptNo) references department(deptNo),  
check (sex in('F', 'M')),  
check (position in('Manager', 'Team Leader','Analyst','Software Developer'))  
);
```

```
-- updating department due to circular foreign key  
alter table department  
add foreign key(mgrEmpNo) references employee(empNo);
```

```
create table project(  
projNo int primary key,  
projName varchar(50) not null,  
deptNo int,  
foreign key(deptNo) references department(deptNo)  
);
```

```
create table worksOn(  
empNo varchar(3),  
projNo int,  
dateWorked date,  
hoursWorked int,  
primary key (empNo, projNo, dateWorked),  
foreign key (empNo) references employee(empNo),  
foreign key(projNo) references project(projNo),  
check (hoursWorked between 0 and 40)  
);
```

```
insert into department values (1, 'it', null);  
insert into department values(2, 'finance', null);  
insert into department values (3, 'hr', null);  
insert into department values (4, 'marketing', null);  
insert into department values (5, 'sales', null);
```

```
insert into department values (6, 'logistics', null);
insert into department values (7, 'legal', null);
insert into department values (8, 'accounting', null);
insert into department values (9, 'operations', null);
insert into department values (10, 'security', null);
insert into department values (11, 'research', null);
insert into department values (12, 'support', null);
```

output:

```
MariaDB [employee_management]> select * from department;
```

```
+-----+-----+-----+
```

```
| deptNo | deptName | mgrEmpNo |
```

```
+-----+-----+-----+
```

```
| 1 | it | E1 |
```

```
| 2 | finance | NULL |
```

```
| 3 | hr | NULL |
```

```
| 4 | marketing | NULL |
```

```
| 5 | sales | E5 |
```

```
| 6 | logistics | E6 |
```

```
| 7 | legal | NULL |
```

```
| 8 | accounting | NULL |
```

```
| 9 | operations | NULL |
```

```
| 10 | security | NULL |
```

```
| 11 | research | E11 |
```

```
| 12 | support | NULL |
```

```
+-----+-----+-----+
```

```
12 rows in set (0.002 sec)
```

-- trigger for constraint :There is only 1 manager for each department

DELIMITER //

create trigger one_manager_per_department

before insert on employee

for each row

begin

if new.position = 'Manager' then

if exists (

select *

from employee

where deptNo = new.deptNo

and position = 'Manager'

) then

signal sqlstate '45000'

set message_text = 'Only one manager per department';

end if;

end if ;

end//

DELIMITER ;

insert into employee values ('E1', 'john', 'doe', '123 main st', '1985-03-15', 'M', 'Manager',
1);

insert into employee values ('E2', 'jane', 'smith', '456 oak ave', '1990-07-22', 'F', 'Analyst',
2);

insert into employee values ('E3', 'bob', 'brown', '789 pine rd', '1988-11-10', 'M', 'Software
Developer', 3);

insert into employee values ('E4', 'alice', 'white', '321 elm st', '1992-05-18', 'F', 'Team Leader', 4);

insert into employee values ('E5', 'mike', 'jones', '654 cedar ln', '1987-09-30', 'M', 'Manager', 5);

insert into employee values ('E6', 'sara', 'davis', '987 birch blvd', '1993-02-14', 'F', 'Manager', 6);

insert into employee values ('E7', 'tom', 'wilson', '111 maple dr', '1986-06-25', 'M', 'Software Developer', 7);

insert into employee values ('E8', 'linda', 'moore', '222 walnut ave', '1991-08-19', 'F', 'Analyst', 8);

insert into employee values ('E9', 'james', 'taylor', '333 spruce st', '1984-12-05', 'M', 'Team Leader', 9);

insert into employee values ('E10', 'emma', 'anderson', '444 pine ave', '1995-04-30', 'F', 'Software Developer', 10);

insert into employee values ('E11', 'paul', 'thomas', '555 oak blvd', '1983-07-11', 'M', 'Manager', 11);

insert into employee values ('E12', 'lucy', 'jackson', '666 elm rd', '1994-01-22', 'F', 'Analyst', 12);

output:

MariaDB [employee_management]> select * from employee;

empNo	fName	lName	address	DOB	sex	position	deptNo
E1	john	doe	123 main st	1985-03-15	M	Manager	1
E10	emma	anderson	444 pine ave	1995-04-30	F	Software Developer	10
E11	paul	thomas	555 oak blvd	1983-07-11	M	Manager	11
E12	lucy	jackson	666 elm rd	1994-01-22	F	Analyst	12
E2	jane	smith	456 oak ave	1990-07-22	F	Analyst	2
E3	bob	brown	789 pine rd	1988-11-10	M	Software Developer	3
E4	alice	white	321 elm st	1992-05-18	F	Team Leader	4

E5	mike	jones	654 cedar ln	1987-09-30	M	Manager	5
E6	sara	davis	987 birch blvd	1993-02-14	F	Manager	6
E7	tom	wilson	111 maple dr	1986-06-25	M	Software Developer	7
E8	linda	moore	222 walnut ave	1991-08-19	F	Analyst	8
E9	james	taylor	333 spruce st	1984-12-05	M	Team Leader	9

+-----+-----+-----+-----+-----+-----+-----+-----+

12 rows in set (0.001 sec)

-- trigger to check that mgrEmpNo are allocated only to employees which are managers

DELIMITER //

create trigger check_manager_assignment

before update on department

for each row

begin

if new.mgrEmpNo is not null then

if not exists (

select *

from employee

where empNo = new.mgrEmpNo

and position = 'Manager') then

signal sqlstate '45000'

set message_text= 'Only employees with position Manager can be assigned as
department manager';

end if;

end if;

end//

DELIMITER ;

```

update department set mgrEmpNo = 'E1' where deptno = 1;
update department set mgrEmpNo = 'E5' where deptno = 5;
update department set mgrEmpNo = 'E6' where deptno = 6;
update department set mgrEmpNo = 'E11' where deptno = 11;

```

output:

```
MariaDB [employee_management]> select * from department;
```

```
+-----+-----+-----+
```

```
| deptNo | deptName | mgrEmpNo |
```

```
+-----+-----+-----+
```

```
| 1 | it | E1 |
```

```
| 2 | finance | NULL |
```

```
| 3 | hr | NULL |
```

```
| 4 | marketing | NULL |
```

```
| 5 | sales | E5 |
```

```
| 6 | logistics | E6 |
```

```
| 7 | legal | NULL |
```

```
| 8 | accounting | NULL |
```

```
| 9 | operations | NULL |
```

```
| 10 | security | NULL |
```

```
| 11 | research | E11 |
```

```
| 12 | support | NULL |
```

```
+-----+-----+-----+
```

```
12 rows in set (0.002 sec)
```

```
insert into project values (1, 'scs', 1);
```

```
insert into project values (2, 'payroll system', 2);
```

```

insert into project values (3, 'hr portal', 3);
insert into project values (4, 'marketing app', 4);
insert into project values (5, 'sales tracker', 5);
insert into project values (6, 'logistics tool', 6);
insert into project values (7, 'legal database', 7);
insert into project values (8, 'accounting system', 8);
insert into project values (9, 'operations hub', 9);
insert into project values (10, 'security monitor', 10);
insert into project values (11, 'research portal', 11);
insert into project values (12, 'support desk', 12);

```

output:

```
MariaDB [employee_management]> select * from project;
```

```

+-----+-----+-----+
| projNo | projName      | deptNo |
+-----+-----+-----+
| 1 | sccs          | 1 |
| 2 | payroll system | 2 |
| 3 | hr portal     | 3 |
| 4 | marketing app  | 4 |
| 5 | sales tracker  | 5 |
| 6 | logistics tool | 6 |
| 7 | legal database | 7 |
| 8 | accounting system | 8 |
| 9 | operations hub | 9 |
| 10 | security monitor | 10 |
| 11 | research portal | 11 |
| 12 | support desk   | 12 |

```


+-----+-----+-----+

```
insert into workson values ('E1', 1, '2024-01-10', 8);
insert into workson values ('E2', 2, '2024-01-11', 6);
insert into workson values ('E12', 1, '2024-01-12', 7);
insert into workson values ('E3', 1, '2024-01-13', 5);
insert into workson values ('E4', 3, '2024-01-14', 4);
insert into workson values ('E5', 1, '2024-01-15', 4);
insert into workson values ('E10', 2, '2024-01-16', 6);
insert into workson values ('E6', 4, '2024-01-17', 7);
insert into workson values ('E7', 5, '2024-01-18', 5);
insert into workson values ('E8', 6, '2024-01-19', 8);
insert into workson values ('E12', 7, '2024-01-20', 6);
insert into workson values ('E11', 8, '2024-01-21', 4);
```

output:

MariaDB [employee_management]> select * from workson;

+-----+-----+-----+-----+

| empNo | projNo | dateWorked | hoursWorked |

+-----+-----+-----+-----+

```
| E1 | 1 | 2024-01-10 | 8 |
| E10 | 2 | 2024-01-16 | 6 |
| E11 | 8 | 2024-01-21 | 4 |
| E12 | 1 | 2024-01-12 | 7 |
| E12 | 7 | 2024-01-20 | 6 |
| E2 | 2 | 2024-01-11 | 6 |
| E3 | 1 | 2024-01-13 | 5 |
```

E4	3	2024-01-14	4
E5	1	2024-01-15	4
E6	4	2024-01-17	7
E7	5	2024-01-18	5
E8	6	2024-01-19	8

+-----+-----+-----+-----+

12 rows in set (0.001 sec)

question 7.25

-- view for projects managed by female managers

create view female_managers_project as

select w.projNo, p.projname, e.fname, e.lname, e.sex, d.mgrEmpno

from department d

inner join employee e

on d.mgrEmpNo = e.empNo

inner join workson w

on e.empNo = w.empNo

inner join project p

on w.projNo = p.projNo

where e.sex = 'F';

output:

MariaDB [employee_management]> select * from female_managers_project

-> order by projNo;

+-----+-----+-----+-----+-----+
projNo projname fname lname sex mgrEmpno
+-----+-----+-----+-----+-----+

```
| 4 | marketing app | sara | davis | F | E6 |
+-----+-----+-----+-----+-----+
1 row in set (0.003 sec)
```

question 7.26

```
-- employee details view
```

```
create view emp_details as
select w.empNo, e.fName, e.lName, p.projName, w.hoursWorked
from employee e
inner join worksOn w
on e.empNo = w.empNo
inner join project p
on w.projNo = p.projNo;
```

output:

```
MariaDB [employee_management]> select * from emp_details;
```

```
+-----+-----+-----+-----+-----+
| empNo | fName | lName | projName | hoursWorked |
+-----+-----+-----+-----+-----+
| E1 | john | doe | sccs | 8 |
| E10 | emma | anderson | payroll system | 6 |
| E11 | paul | thomas | accounting system | 4 |
| E12 | lucy | jackson | sccs | 7 |
| E12 | lucy | jackson | legal database | 6 |
| E2 | jane | smith | payroll system | 6 |
| E3 | bob | brown | sccs | 5 |
```

E4	alice	white	hr portal	4
E5	mike	jones	sccs	4
E6	sara	davis	marketing app	7
E7	tom	wilson	sales tracker	5
E8	linda	moore	logistics tool	8

+-----+-----+-----+-----+-----+

12 rows in set (0.002 sec)

Question 7.27

```
create view empproject(empno, projno, totalhours)
as select w.empno, w.projno, sum(hoursworked)
from employee e, project p, workson w
where e.empno = w.empno and p.projno = w.projno
group by w.empno, w.projno;
```

output:

```
MariaDB [employee_management]> select * from empproject;
```

+-----+-----+-----+
empno projno totalhours
+-----+-----+-----+
E1 1 8
E10 2 6
E11 8 4
E12 1 7
E12 7 6
E2 2 6
E3 1 5

E4	3	4
E5	1	4
E6	4	7
E7	5	5
E8	6	8

```
+-----+-----+-----+
```

12 rows in set (0.004 sec)

```
MariaDB [employee_management]> select count(projNo)
-> from empproject
-> where empNo = 'E1';
```

count(projNo)

```
+-----+
```

1

```
+-----+
```

1 row in set (0.003 sec)

output:

```
MariaDB [employee_management]> select empNo, sum(totalhours) as totalHours
-> from empproject
-> group by empNo;
```

empNo totalHours

```
+-----+-----+
```

E1	8
E10	6

E11	4
E12	13
E2	6
E3	5
E4	4
E5	4
E6	7
E7	5
E8	8

+-----+-----+

11 rows in set (0.002 sec)

Question 7.28

Materialized View Maintenance of ExpensiveParts

A materialized view physically stores the results of a query unlike a regular view which is virtual. To maintain the ExpensiveParts materialized view, any changes made to the base table Part must be reflected in the stored results.

The view contains distinct part numbers where partCost > 1000. Since a part can have different prices under different contracts, maintenance depends on the type of change made to the Part table.

1. INSERT

- if new partCost > 1000, add partNo to view (no need to access base table)
- if new partCost ≤ 1000, do nothing (no need to access base table)

2. UPDATE

- if partCost updated above 1000, add partNo to view (no need to access base table)
- if partCost updated below 1000, MUST access base table to check if partNo still exists under another contract with partCost > 1000

3. DELETE

- MUST access base table to check if partNo still exists under another contract with partCost > 1000 before removing from view

The materialized view can be maintained without accessing the base table Part only during INSERT operations or when a cost is increased above 1000. For updates below 1000 and deletions, the base table must always be checked because one part can have multiple contracts with different prices.